

DSL Symbolic Visual Guide

Quick Reference for Claude Code Agent Orchestration ---



Core Symbols

• Agent	Operator	== State/Context
⦿ Skill	Data flow	... Optional path
▣ Command	Async	— Synchronization barrier
○ Workflow	/ Conditional	→ Loop/iteration

Operators at a Glance

→ Sequential	$A \rightarrow B \rightarrow C$
Parallel	$A \parallel B \parallel C$
⊕ Combination	$A \oplus skill_1 \oplus skill_2$
: Binding	<code>agent : "task"</code>
= Assignment	<code>param = value</code>
? Conditional	$A \rightarrow ? \rightarrow B \mid C$
* Iteration	$(A \rightarrow B)^*$
↺ Retry	$A \circ^3 \rightarrow fallback$

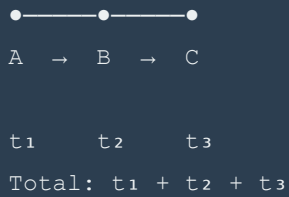
Level 1: Single Invocation



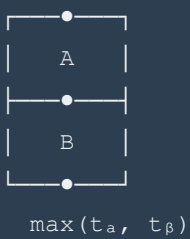
Example: `api-architect : "design API" ---`

Level 2: Binary Operations

Sequential ($\rightarrow$)



Parallel ($\parallel$)

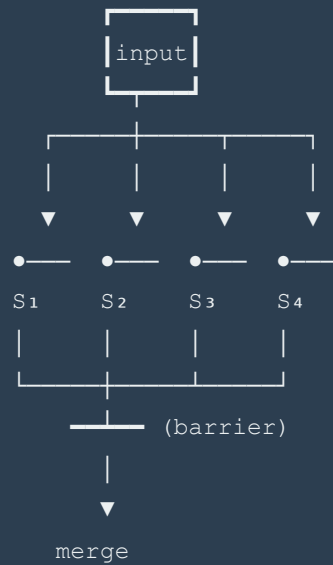


Combination ($\oplus$)

• $\oplus$ • $\oplus$ •

```
agent + skill1 + skill2  
= enhanced_agent
```

Level 3: Parallel Streams



Example: (A || B || C) : task ---

Level 4: Complex Orchestration

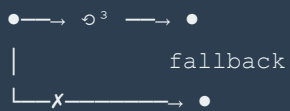
Fan-Out / Fan-In



Conditional Branch



Error Handling

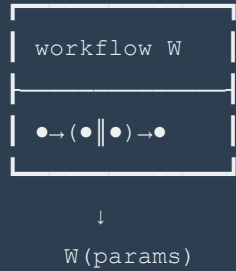


Nested Parallel

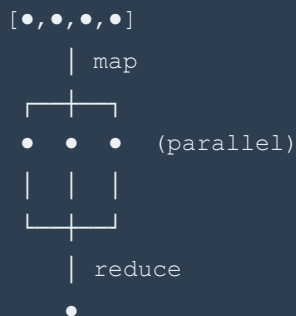


Level 5: Workflow Composition

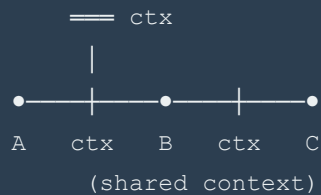
Named Workflow



Map-Reduce



Context Sharing



Level 6: Meta-Programming

Functor

```
Workflow{A}
  |
  fmap(f)
  |
  ▼
Workflow{B}
```

Applicative



Monad

```
do {
  a ← •
  b ← •(a)
  c ← •(b)
  return c
}
```

Workflow Generator

```
meta(T) → O
|
├→ O(User)
├→ O(Product)
└→ O(Order)
```

Composition Patterns

Pipeline

```
● → ● → ● → ●
linear flow
```

Diamond



Star (Hub-Spoke)



Tree

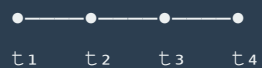


Mesh



Time & Token Diagrams

Sequential



Total: $\Sigma(t_i)$

Tokens: $\Sigma(\text{tok}_i)$

Parallel



Total: $\max(t_i) + \text{merge}$
Tokens: $\max(\text{tok}_i) + \text{overhead}$

Hybrid



Total: $\text{seq} + \max(\text{par})$

State Transitions

Simple

$S_0 \rightarrow [\bullet] \rightarrow S_1$

Fork/Join



Conditional

```
S0 → ? → S1  
    ↳ S2
```

Loop

```
S0 → • ↺ S0  
    ↓  
    S1
```

Error Patterns

Retry

```
•—X—○—•—X—○—•—X—→ fallback  
1st  retry 2nd  retry 3rd
```

Circuit Breaker

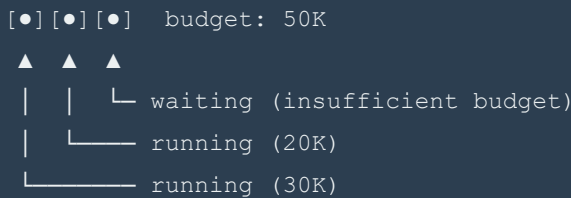
```
CLOSED —failures→ OPEN  
    ↑                |  
    |                timeout  
    |                ↓  
    ↳success— HALF_OPEN
```

Graceful Degradation

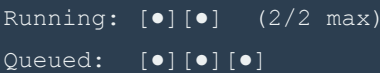


Resource Constraints

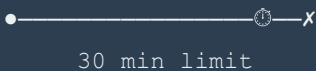
Budget Limit



Concurrency Limit



Timeout



DAG Patterns

Linear DAG

```
•→•→•→•  
Depth: 4  
Width: 1
```

Balanced Tree DAG

```
  •  
 / \  
•   •  
/ \ / \  
• • • •  
Depth: 3  
Width: 4
```

Layered DAG

```
Layer 0:  • • •  
          | \ / | /  
Layer 1:  • • •  
          | X |  
Layer 2:  • •  
          |/  
Layer 3:  •
```

Type Signatures

Sequential

```
•A → •B : A → B → workflow{A,B}
```

Parallel

```
•A || •A : A → (B,C) → workflow{A, (B,C)}
```

Generic

```
○{T} : ∀T. T → Result{T}
```

Quick Pattern Lookup

| Pattern | Symbol | Use When | |-----|-----|-----| | Pipeline |  | Dependent steps |
| Fan-out |  | Independent parallel | | Conditional |  | Runtime decisions | | Loop |
 | Retry/iterate | | Map-reduce |  | Collection processing | | Context share |  |
Shared resources | | Fallback |  | Error recovery | | Barrier |  | Synchronization | ---

Common Compositions

Research Pipeline

```
•→(•||•||•)→•
research  parallel  synthesize
          analysts
```

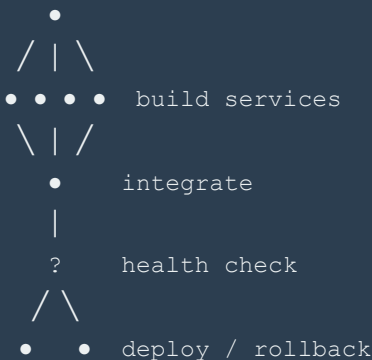
CI/CD

```
•→•→?→•→•
build  \
test   deploy
check  /
       rollback
```

Data Processing

```
[•,•,•,•]
  | split
•||•||•  transform
  | reduce
  • aggregate
```

Microservice Deploy



Legend Summary

SHAPES

- Agent
- Skill
- ▣ Command
- Workflow

CONNECTIONS

- Sequential
- || Parallel
- = State/Context
- ⋮ Async flow
- / Branch
- Barrier

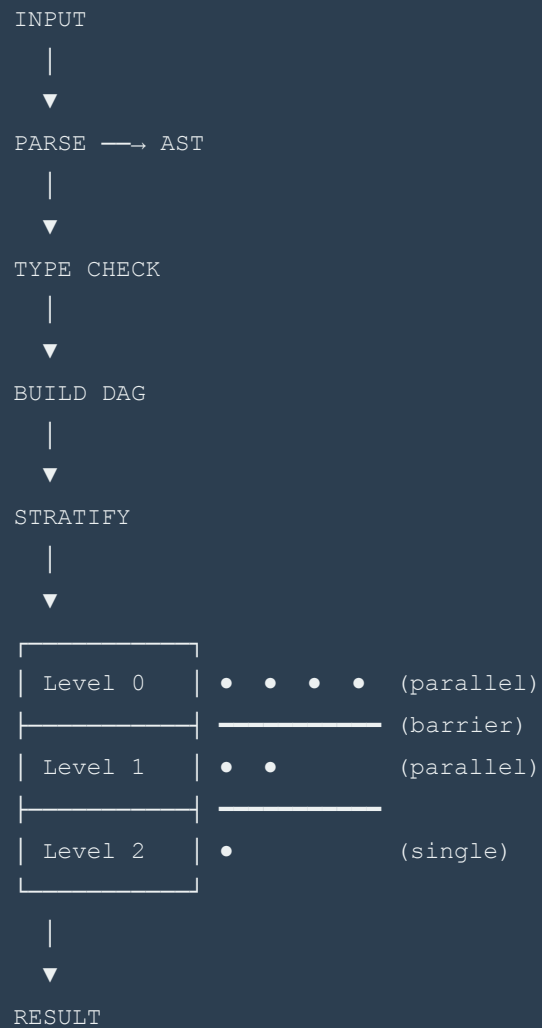
MODIFIERS

- ↺ Retry
- ? Conditional
- * Iteration
- ✖ Error/Failure
- ⌚ Timeout
- ✕ Intersection

Complexity Quick Reference

L1:	•	(5K tok, 5 min)
L2:	•→• • •	(15K tok, 10 min)
L3:	• • •	(50K tok, 30 min)
L4:	•→(• •)→?	(100K tok, 60 min)
L5:	○(•→• •)	(200K tok, 2 hr)
L6:	meta{T}	(500K+ tok, 4+ hr)

Execution Model



Mathematical Foundations

CATEGORY THEORY

$\bullet \rightarrow \bullet$: morphism

$\text{id} \rightarrow \bullet = \bullet$

$(f \circ g) \circ h = f \circ (g \circ h)$

MONOID

$\bullet \oplus \bullet$: composition

$\bullet \oplus \emptyset = \bullet$

$(\bullet \oplus \bullet) \oplus \bullet = \bullet \oplus (\bullet \oplus \bullet)$

FUNCTOR

$\text{fmap}: (a \rightarrow b) \rightarrow (F\ a \rightarrow F\ b)$

$F(g \circ f) = F(g) \circ F(f)$

GRAPH THEORY

$V = \{\text{agents}\}$

$E = \{\text{dependencies}\}$

$\text{DAG} = (V, E)$

TOPOLOGICAL SORT

$\text{in-degree} \rightarrow 0$

↓

schedule

↓

execute

↓

result

--- End of Symbolic Visual Guide Use this as a quick reference alongside the comprehensive documentation